Frame to frame interpolation for high-dimensional data visualisation using the woylier package

by Zoljargal Batsaikhan, Dianne Cook, and Ursula Laa

Abstract The woylier package implements tour interpolation paths between frames using Givens rotations. This provides an alternative to the geodesic interpolation between planes currently available in the tourr package. Tours are used to visualise high-dimensional data and models, to detect clustering, anomalies and non-linear relationships. Frame-to-frame interpolation can be useful for projection pursuit guided tours when the index is not rotationally invariant. It also provides a way to specifically reach a given target frame, useful for some applications. We demonstrate the method for exploring non-linear relationships between currency cross-rates.

1 Introduction

When data has up to three variables, visualization is relatively intuitive, while with more than three variables, we face the challenge of visualizing high dimensions on 2D displays. This issue was tackled by the *grand tour* (Asimov, 1985), which can be used to view data in more than three dimensions using linear projections. It is based on the idea of rotations of a lower-dimensional projection in high-dimensional space. The grand tour allows users to see dynamic low-dimensional (typically 2D) projections of higher-dimensional space. Originally, Asimov's grand tour presented the viewer with an automatic movie of projections with no user control. Since then, new work has added interactivity to the tour, giving more control to users (Buja et al., 2005). New variations include the manual (Cook and Buja, 1997) or radial tour (Laa et al., 2023), little tour, guided tour (Cook et al., 1995), local tour, and planned tour. These are different ways of selecting the sequence of projection bases for the tour; see Lee et al. (2022).

The guided tour combines projection pursuit with the grand tour, and it is implemented in the **tourr** package (Wickham et al., 2011). Projection pursuit, originally proposed in Kruskal (1969), is a procedure used to locate the projection of high-to-low dimensional space that should expose the most interesting feature of the data. It involves defining a criterion of interest, a numerical objective function that indicates the interestingness of each projection, and an optimization for selecting planes with increasing values of the function. A number of such criteria have been developed based on clustering, spread, and outliers and made available in R packages. The **tourr** contains the indexes defined in Cook et al. (1993) that can be used for clustering, outliers, and skewness, and is the only package making the guided tour available. Large data versions of these can be found in the package **PPbigdata** (Duan and Cabrera, 2024). The package **pursuit** (Ossani and Cirillo, 2026) also contains these indexes and several more. The **PPCI** (Hofmeyr and Pavlidis, 2019) does projection pursuit as part of a cluster analysis, and the **ppgmmga** (Scrucca and Serafini, 2019) provides an index for clustering based on Gaussian mixture models. The **REPPlab** (Fischer et al., 2021) provides indexes for detecting clusters and outliers.

A tour path is a sequence of projections, and we use an interpolation to produce small steps simulating a smooth movement. The current implementation of tour in the **tourr** package uses geodesic interpolation between planes. The geodesic interpolation path is the shortest path between planes with no within-plane spin (see Buja et al. (2004) for more details). As a result, the rendered target plane could be a within-plane rotation of the target plane originally specified. This is not a problem when the structure we are looking for can be identified from any rotation. However, even simple associations in 2D, such as the calculated correlation between variables, can be very different when the basis is rotated.

Most projection pursuit indexes, particularly those provided by **tourr**, are rotationally invariant. However, there are some applications where the orientation of frames does matter. One example is the splines index proposed by Grimm (2016). The splines index computes a spline model for the two variables in a projection (using the implementation in **mgcv** (Wood, 2011)), in order to measure non-linear association. It compares the variance of the response variable to the variance of residuals, such that the functional dependence is stronger when the index value is larger. It can be useful to detect non-linear relationships in high-dimensional data. However, its value will change substantially if the projection is rotated within the plane (Laa and Cook, 2020). The procedure in Grimm (2016) is less affected by the orientation because it considers only pairs of variables, and it selects the maximum value found when exchanging which variable is considered as a predictor or response variable.

Figure 1 illustrates the rotational invariance problem for a modified splines index, where we

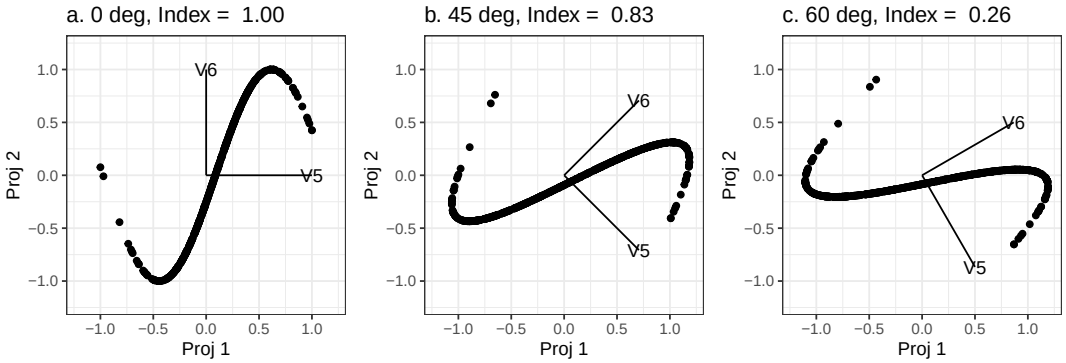

Figure 1: The impact of rotation on a spline index that is NOT rotation invariant. The index value for different within-plane rotations takes very different values: (a) the original projection has a maximum index value of 1.00, (b) axes rotated 45^o drops the index value to 0.83, (c) axes rotated 60^o drops the index to a very low 0.26. Geodesic interpolation between planes will have difficulty finding the maximum of an index like this because it is focused only on the projection plane, not the frame defining the plane.

always consider the horizontal direction as the predictor variable and the vertical direction as the response. Thus, our modified index computes the splines on one orientation, exaggerating the rotational variability. The example data was simulated to follow a sine curve, and the modified splines index is calculated on different within-plane rotations of the data. Although they have the same structure, the index values vary greatly.

The lack of rotation invariance of the splines index raises complications in the optimization process in the projection-pursuit guided tour as available in **tourr**. Fixing this is one motivation of this work, and it makes the explanation of frame-to-frame interpolation easier. The goal with the frame-to-frame interpolation is that optimization would find the best within-plane rotation, and thus appropriately optimize the index.

We should note that if optimization were the only goal, it could be achieved simply by modifying the index to also optimize across all frames in the same plane by adding a within-plane rotation and index calculation. This might slow the optimisation down a little, but it would achieve the objective of finding the best projection, regardless of orientation. However, there are some specific applications where we would like to reach a specific frame at the target. This is an area of potential future research, but places where we have found it to be desirable are interpolating between frames provided by two different principal component solutions, perhaps computed on two different samples, to digest the similarity and difference between them. Another recent example is to examine high-dimensional ternary diagrams (simplexes) summarizing preferential data, where we would like to rotate through very specific frames corresponding to pairs or preferences to understand how preferences might flow between groups.

A few alternatives to geodesic interpolation were proposed by Buja et al. (2005), including the decomposition of orthogonal matrices, Givens decomposition, and Householder decomposition. The purpose of the **woylier** package is to implement the Givens paths method in R. This algorithm adapts the Givens matrix decomposition technique, which allows the interpolation to be between frames rather than planes.

This article is structured as follows. The next section provides the theoretical framework of the Givens interpolation method, followed by a section about the implementation in R. The method is applied to search for non-linear associations between currency cross-rates.

2 Background

The tour method of visualization shows a movie that is an animated high-to-low dimensional data rotation. It is a one-parameter (time) family of static projections. Algorithms for such dynamic projections are based on the idea of smoothly interpolating a discrete sequence of projections (Buja et al., 2005).

The topic of this article is the construction of the paths of projections. The interpolation of these paths can be compared to connecting line segments that interpolate points in Euclidean space. Interpolation acts as a bridge between a continuous animation and the discrete choice of sequences of projections.

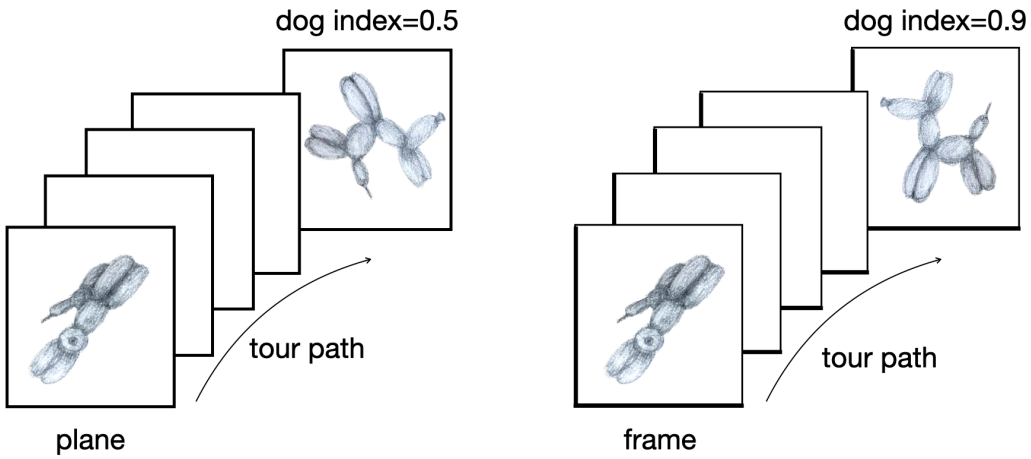

Figure 2: Plane to plane interpolation (left) and frame to frame interpolation (right). We use the dog index for illustration purposes. Orientation of the data within the plane could affect the index value. Frame-to-frame interpolation guarantees reaching a particular frame of the many defining a plane.

Interpolating paths of planes versus paths of frames

The **tourr** package implements geodesic interpolation between planes, and the final interpolation step will reach the rotation of the target frame, avoiding any within-plane spin along the path. When the orientation of projections matters, interpolation between frames is required. The orientation of the frames could be important when a non-linear projection pursuit index function is used in the guided tour. This is illustrated by the different index values shown in the sketch in Figure 2, as well as the splines index for the sine curve in Figure 1.

To describe the interpolation algorithms, we will use the following notation.

- Let p be the dimension of the original data and d be the dimension onto which the data is being projected.
- A frame F is defined as a $p \times d$ matrix with pairwise orthogonal columns of unit length that satisfies
$$F^T F = I_d,$$
where I_d is the identity matrix in d dimensions.
- Paths of frames are given by continuous one-parameter families $F(t)$ where $t \in [a, z]$ represents time. We denote the starting frame (at time a) by $F_a = F(a)$ and target frame (at time z) by $F_z = F(z)$. Usually, F_z is the target frame that has been chosen according to the selected tour method. While a grand tour chooses target frames randomly, the guided tour chooses the target frame by optimizing the projection pursuit index. Interpolation methods are used to find the path that moves from F_a to F_z.

Preprojection algorithm

In order to make the interpolation algorithm simple, we carry out a preprojection step to find the subspace that the interpolation path, $F(t)$, is traversing. In other words, the preprojection step defines the joint subspace of F_a and F_z and makes sure the interpolation path is limited to that space.

The procedure starts with forming an orthonormal basis by applying Gram-Schmidt to F_z with regard to F_a: $F_\star = F_z - F_a(F_a^\top F_a)^{-1}F_a F_z$, followed by orthonormalizing subsequent columns of $F_\star$ against all preceding columns. (The function `orthonormalise_by()` in the **tourr** does this.) Then we build the preprojection basis B by combining F_a and $F_\star$ as follows:

$$B = (F_a, F_\star)$$

The dimension of the resulting orthonormal basis, B, is $p \times 2d$.

Then, we can express the original frames in terms of this basis:

$$F_a = BW_a, F_z = BW_z$$

The interpolation problem is then reduced to the construction of paths of frames $W(t)$ that interpolate between the preprojected frames W_a and W_z. By construction, W_a is a $2d \times d$ matrix of

1s and 0s. This is an important characteristic of our interpolation algorithm of choice, the Givens interpolation.

Givens interpolation path algorithm

A rotation matrix is a transformation matrix used to perform a rotation in Euclidean space. The matrix that rotates a 2D plane by an angle θ looks like this:

$$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

If the rotation is in the plane of two selected variables, it is called a Givens rotation. Let's denote those 2 variables as i and j. The Givens rotation is used for introducing zeros, for example, when computing the QR decomposition of a matrix in linear algebra problems.

The interpolation method in the **woylier** package is based on the fact that in any vector of a matrix, one can zero out the i-th coordinate with a Givens rotation in the (i, j)-plane for any $j \neq i$ (Golub and Loan, 2013). This rotation affects only coordinates i and j and leaves all other coordinates unchanged. Sequences of Givens rotations can map any orthonormal d-frame F in p-space to the standard d-frame

$$E_d = ((1,0,0,...)^T, (0,1,0,...)^T, ...).$$

The resulting interpolation path construction algorithm from starting frame F_a to target frame F_z is illustrated below. The example is for $p = 6$ and $d = 2$.

1. Construct preprojection basis B by orthonormalizing F_z with regards tp F_a with Gram-Schmidt (same procedure as described above).

In our example, F_a and F_z are $p \times d$ or 6×2 matrices that are orthonormal. The preprojection basis B is $p \times 2d$ matrix that is 6×4.

2. Get the preprojected frames using the preprojection basis B.

$$W_a = B^T F_a = E_d$$

and

$$W_z = B^T F_z$$

In our example, W_a looks like:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

W_z is an orthonormal $2d \times d$ matrix that looks like:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \\ a_{41} & a_{42} \end{bmatrix}$$

3. Then, we can construct a sequence of Givens rotations that maps W_z to W_a with such angles that make one element zero at a time:

$$W_a = R_m(\theta_m)...R_2(\theta_2)R_1(\theta_1)W_z$$

At each rotation, the angle θ_i that zeros out the next coordinate of a plane is calculated. Here $m = \sum_{k=1}^{d}(2d - k)$, so when $d = 2$ we need $m = 5$ rotations with 5 different angles, each making one element 0. For example, the first rotation angle θ_1 is an angle in radians between $(1, 0)$ and (a_{11}, a_{21}). This rotation matrix would make element a_{21} zero:

$$R_1(\theta_1) = G(1, 2, \theta_1) = \begin{bmatrix} cos\theta_1 & -sin\theta_1 & 0 & 0 \\ sin\theta_1 & cos\theta_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Here $G(i, j, \theta_k)$ denotes a Givens rotation in components i and j by angle θ_k. In the same way, we zero out the elements a_{31} and a_{41}. Because of the orthonormality this means that now $a_{11} = 1$ and that $a_{12} = 0$. We thus need only two more rotations to zero out a_{32} and a_{42}.

Each θ_i is an angle in 2D and is computed from the polar coordinates returned by the `atan2()` function.

4. The inverse mapping is obtained by reversing the sequence of rotations with the negative of the angles; we start from the starting basis and end at the target basis.

$$R(\theta) = R_1(-\theta_1)...R_m(-\theta_m),\ W_z = R(\theta)W_a$$

Performing these rotations would go from the starting frame to the target frame in one step, but we want to do it sequentially in a number of steps so that interpolation between frames looks dynamic.

5. Next, we include the time parameter, t, so that the interpolation process can be rendered in the movie-like sequence. We break each θ_k into the number of steps, n_{step}, that we want to take from the starting frame to the target frame, which means it moves by an equal angle in each step. Here, n_{step} should vary based on the angular distance between F_a and F_z, such that when watching a sequence of interpolations, we have a fixed angular speed.
6. Finally, we reconstruct our original frames using B. This reconstruction is done at each step of interpolation so that we have the interpolated path of frames as the result.

$$F_t = BW_t$$

At each time t, we can project the data using the frame F_t.

3 Implementation

We implemented each of the steps in the Givens interpolation path algorithm in separate functions and combined them into a single function `givens_full_path()` to produce the full set of F_t. The same functions are used to integrate the Givens interpolation with the `animate()` functions of the **tourr** package. Table 1 lists the input and output of each function and its descriptions. Functions to use with `animate()` are described separately below.

When using **tourr**, we typically want to run a tour live, such that target selection and interpolation are interleaved, and the display will show the data for each frame F_t in the interpolation path. The implementation in **tourr** was described in Wickham et al. (2011), and with **woylier** we provide functions to use the Givens interpolation with the grand tour, guided tour and planned tour. To do this, we rely primarily on the function `givens_info()` which calls the functions listed in the Table above and collects all necessary information for interpolating between a given starting and target frame. The function `givens_path()` then defines the interpolation and can be used instead of `tourr::geodesic_path()`. Wrapper functions for the different tour types are available to use this interpolation, since in the `tourr::grand_tour()` and other path functions, this is fixed to use the geodesic interpolation. Calling a grand tour with Givens interpolation for direct animation will then use:

```
tourr::animate_xy(<data>, tour_path = woylier::grand_tour_givens())
```

4 Comparison of geodesic interpolation and Givens interpolation

The `givens_full_path()` function returns the intermediate interpolation step projections for a given number of steps. The code chunk below demonstrates the interpolation between 2 random bases in 5 steps.

```
givens_full_path(base1, base2, nsteps = 5)
```

To compare the path generated with the Givens interpolation to that found with geodesic interpolations, we look at the rotation of the sine data shown in Figure 1. We consider a subset in $p = 4$ dimensions where the first two dimensions contain noise and the last two contain the sine curve. Starting from a random projection, we want to interpolate towards the original sine curve. The path comparison is shown in Figure 3.

Plotting the interpolated paths

For further comparison and to check that the interpolation is moving in equally sized steps, we directly plot the interpolated paths. The space of 1D projections defines a unit sphere, while

Table 1: Primary functions in the woylier package.

name	description	input	output
`givens_full_path(Fa, Fz, nsteps)`	Construct full interpolated frames.	Starting and target frame (Fa, Fz) and number of steps	An array with nsteps matrix. Each matrix is interpolated frame in between starting and target frames.
`preprojection(Fa, Fz)`	Build a d-dimensional pre-projection space by orthonormalizing Fz with regard to Fa.	Starting and target frame (Fa, Fz)	B pre-projection p x 2D matrix
`construct_preframe(Fa, B)`	Construct preprojected frames.	A frame and the pre-projection p x 2D matrix	Pre-projected frame in pre-projection space
`row_rot(a, i, k, theta)`	Performs Givens rotation .	A frame and the pre-projection p x 2D matrix	theta angle rotated matrix a
`calculate_angles(Wa, Wz)`	Calculate angles of required rotations to map Wz to Wa.	Preprojected frames (Wa, Wz)	Names list of angles
`construct_moving_frame(Wt, B)`	Reconstruct interpolated frames using pre-projection.	Pre-projection matrix B, Each frame of Givens path	A frame of on a step of interpolation

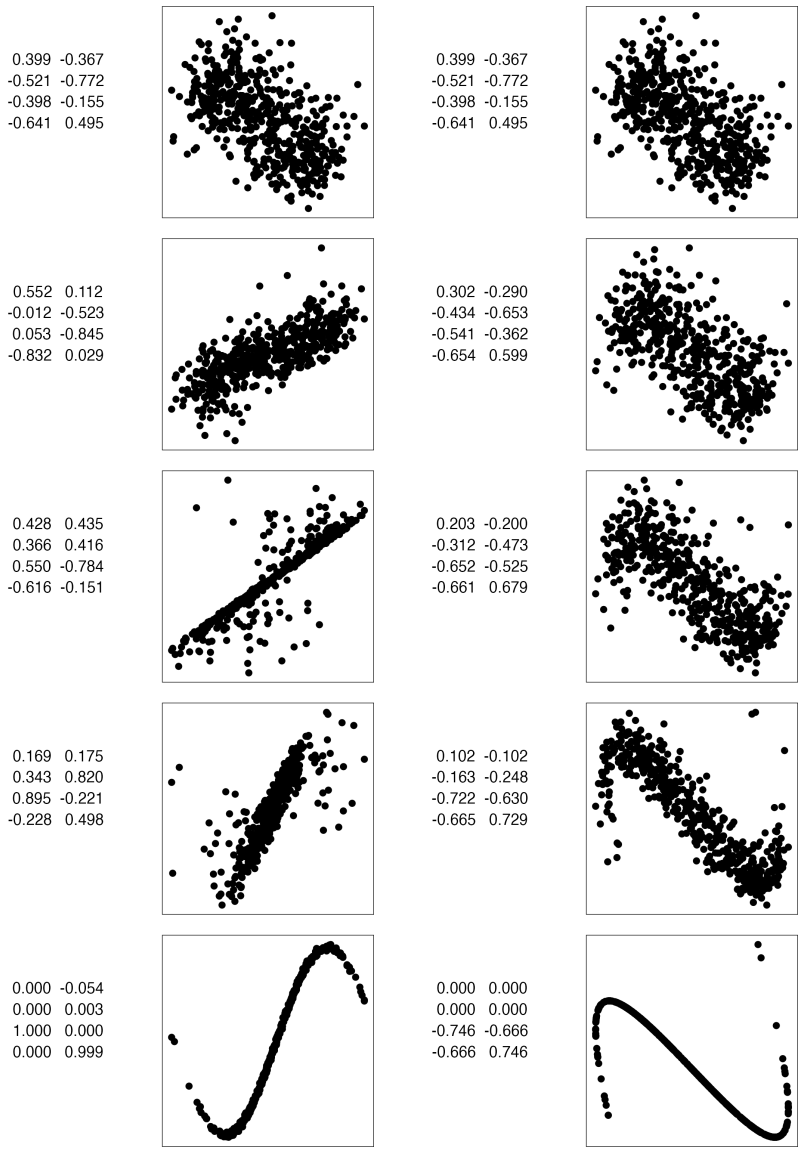

Figure 3: Comparison of Givens path and geodesic path between 2D projections. The Givens path preserves the frame ending at the provided basis (frame), while geodesic is agnostic to the particular basis. In general, the geodesic is preferred because it removes within-plane spin, but occasionally it is helpful to very specifically arrive at the prescribed basis.

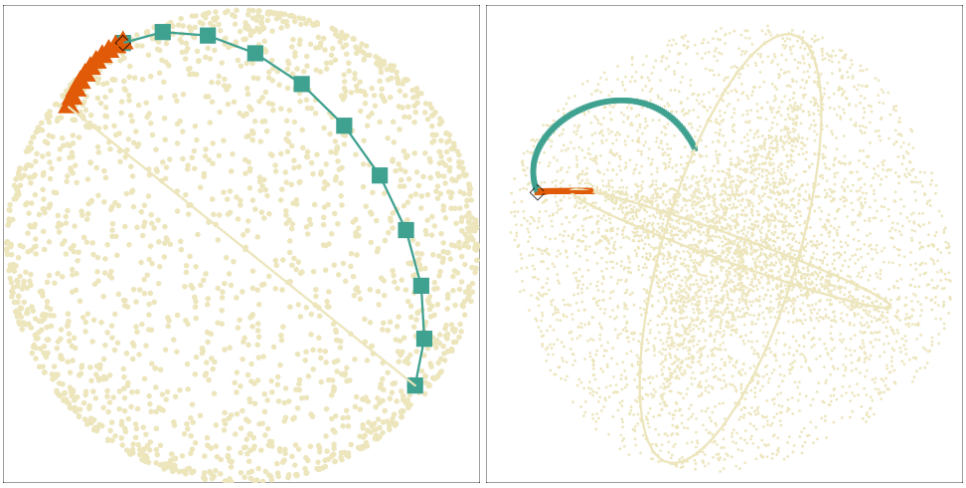

Figure 4: Interpolation steps of 1D (left) and 2D (right) projections of 3D data made with a Givens path (forest green square) and a geodesic path (dark orange triangle). The black diamond indicates the starting basis common to both interpolations. The cream points represent the space of all projections, which is a sphere for 1D projections and a torus for 2D projections. In the 1D example, geodesic arrives at the opposite side of the sphere to Givens, indicating that it has flipped the direction of the vector in order to make the shortest path to the same plane. A similar thing happens for the 2D example, geodesic flips the sign of one basis vector, but it defines the same plane, as indicated by the cream circles.

2D projections define a torus. To illustrate the space, points on the surface of the sphere and the torus shape are randomly generated by functions from the **geozoo** package (Schloerke, 2016). The interpolated paths are then compared within that space.

A 1D projection of data in p dimensions corresponds to a linear combination where the weights are normalized. Therefore, we can plot the point on the surface of a hypersphere. In this case, the Givens interpolation will reach the exact point, while geodesic interpolation might flip the direction and reach a point on the opposite side of the hypersphere. Figure 4 (left) shows the comparison of the interpolation steps using the same target plane, for an example with $p = 3$. Because the flipped target is close to the starting plane, the geodesic path is a lot shorter.

In the case of 2D projections, we can plot the interpolated path between 2 frames on a torus. A torus can be seen as a crossing of 2 circles that are orthonormal, as is the case with our projection onto 2D. Figure 4 (right) compares the interpolation paths for $p = 3$.

For a 2D projection, the same target plane is found when rotating the basis within the plane or when reflecting across one of the two directions (the reflected basis can then also be rotated). This means the space of target bases is constrained to two circles on the torus, and these are disconnected because the reflection corresponds to a jump. In the high-dimensional space, we can imagine the reflection as flipping over the target plane, resulting in a reflection of the normal vector on the plane. While the Givens interpolation will reach the exact basis, a geodesic interpolation towards the same target plane can land anywhere along those two circles, depending on the starting basis in the interpolation.

5 Application

This section describes the application of the Givens interpolation path with a guided tour to explore non-linear association in multivariate data. We use cross-rates for currencies relative to the US dollar. A cross-rate is an exchange rate between two currencies computed by reference to a third currency, usually the US dollar. A strong non-linear functional relationship would indicate that underlying the collection of cross-currency rates is a single latent factor explaining all the movement in that time period.

The data were extracted from Rates (2026) and contain cross-rates for ARS, AUD, EUR, JPY, KRW, MYR between 2019-11-1 and 2020-03-31. Figure 5 shows how the currencies changed relative to USD over the time period. We see some collective behavior in March of 2020 with EUR and JPY increasing in a similar manner, and smaller currencies decreasing in value. This could be understood as a consequence of flight-to-quality at these uncertain times.

We are interested in capturing this relation over time, and from the time series visualization,

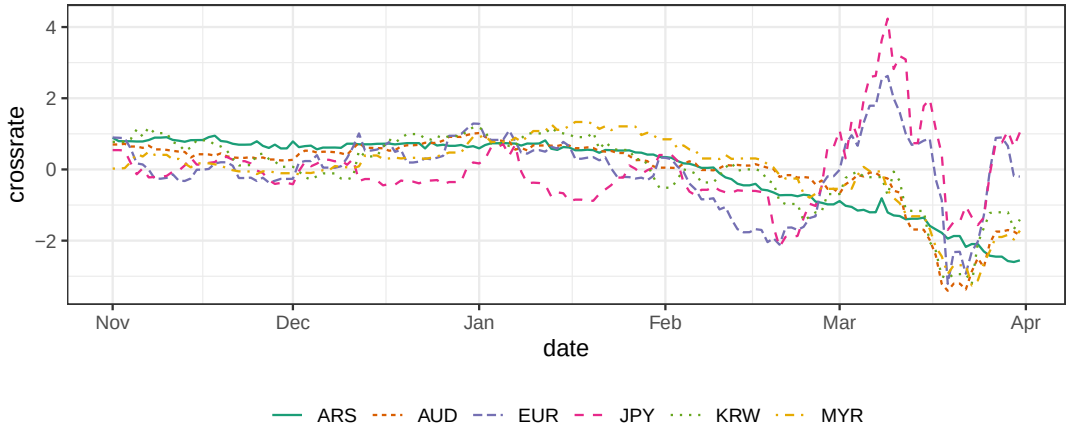

Figure 5: Examining the behaviour of six cross-currency rates (ARS, EUR, KRW, AUD, JPY, MYR) prior to and in the first month of the pandemic. All the currencies are standardised, and the sign is flipped. JPY and EUR strengthened against the USD (high values) in March, while the other currencies weakened (low values).

we expect that we can capture the main dynamics in a two-dimensional projection from the six-dimensional space of currencies. Thus, we start from $p = 6$ (the different currencies) and $n = 152$, the number of days in our sample. We expect that a projection onto $d = 2$ dimensions should capture the relation between the two groups of currencies mentioned above, and this should be identified within the noise of the random fluctuations.

We may expect that we can capture the dependence between the two groups using principal components analysis (PCA). Figure 6 shows a scatter plot matrix of the principal components (PCs) of our dataset. Indeed, we find a strong non-linear association between the first two PCs. Investigation of the rotation shows that the first PC is primarily a balanced combination between ARS, AUD, KRW, and MYR (and a smaller contribution of EUR), and the second contribution is dominated by EUR and JPY, contrasted with smaller contributions from ARS and MYR. Our next step is to use projection pursuit to further explore for non-linear functional dependence.

Guided tour optimization

We now use the splines index to further explore functional dependence beyond what is observed from PCA. Note here that because of strong linear correlations between the currencies, we start from the first four PCs, explaining over 97% of the variance in the data. To make the challenge of finding functional dependence harder, we start from a projection onto the third and fourth PCs. We hope to find a stronger functional dependence than uncovered by PCA. The `display_pca()` function in **tourr** is used to show the original variables that correspond to the 2D projection of the first four PCs, so that interpretation can be made with respect to the cross-currencies.

There are several options for optimization in the **tourr** package. Geodesic optimization (GEOO) (provided by the `search_geodesic()` function) is a derivative-free method approximating a stochastic optimization algorithm. From any starting plane, a random search in the near neighborhood delivers the most promising direction to follow. The tour using geodesic interpolation follows this direction until the index value decreases, and then a new direction search is conducted. It guarantees that the index value always increases, but it can be slow, and it can get trapped at local maxima. Typically, the best method is simulated annealing (provided by the `search_better()` function), which performs a random search of a large neighborhood and then cools down the neighborhood size as the optimization progresses. We have used this approach, inserting geodesic (SAGEO) and Givens interpolation (SAGIV) to move between targets.

The results are shown in Figure 7 and we see on the right that the guided tour with SAGIV has identified a non-linear functional dependence between the x and y axes in the final projection. The result on the left is using GEOO, and the result in the middle uses SAGEO. Since these methods do not allow for within-plane rotation, they were not able to identify the same and best functional relationship even though they are using the same starting plane and index function.

To understand the result in more detail, we use tools from **ferrn** (Zhang et al., 2021) to show how the index value is changing over the optimization in Figure 8. Points indicate selected target planes and are connected by lines showing the index value along the interpolated path between them. For ease of comparison, the horizontal line shown in all three panels indicates the maximum index value found when using Givens interpolation.

The top row shows the path found via GEOO. This search strategy only moves along geodesic

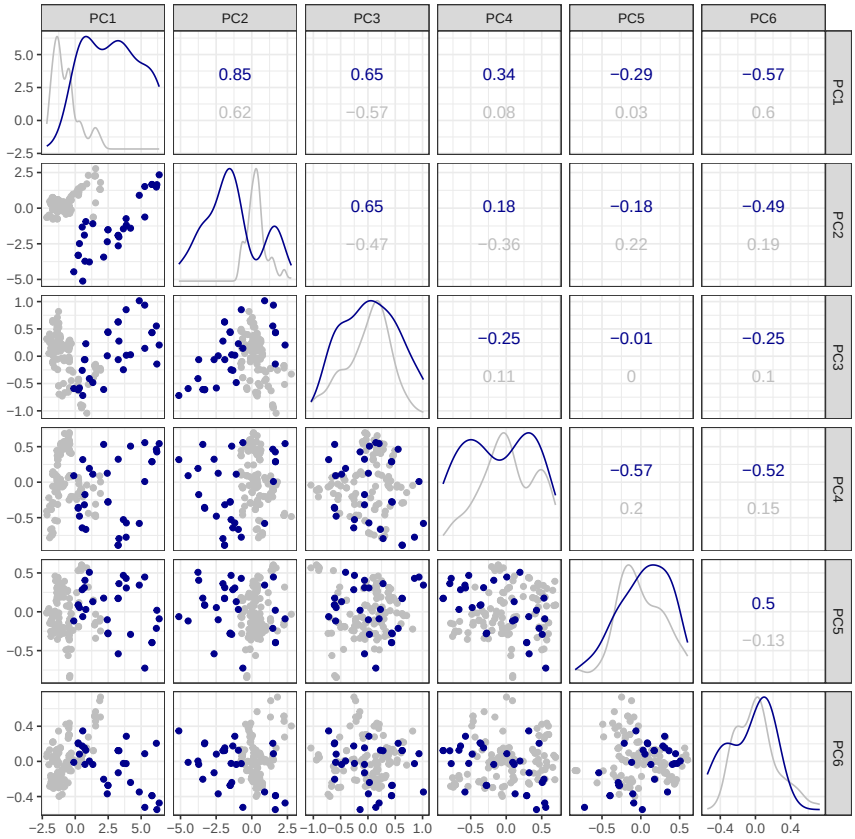

Figure 6: Scatterplot matrix of the six principal components. Observations in March 2020 are highlighted in dark blue; all other months are shown in grey. There is a strong non-linear dependence between PC1 and PC2.

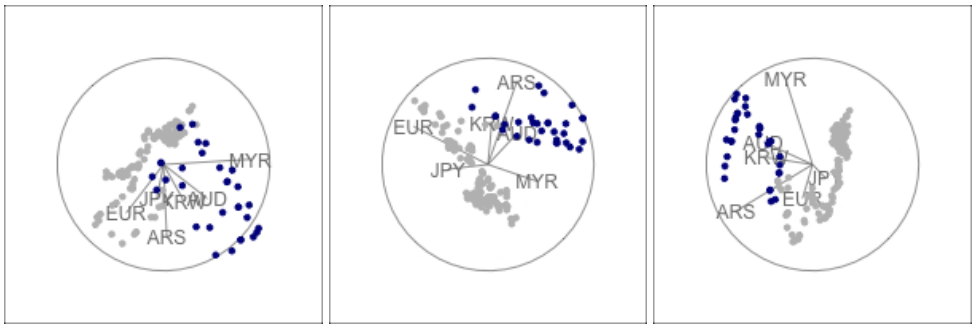

Figure 7: Final view after optimization in the guided tour using geodesic optimization (GEOO) (left), simulated annealing with geodesic interpolation (SAGEO) (middle) and simulated annealing with Givens interpolation (SAGIV) (right).

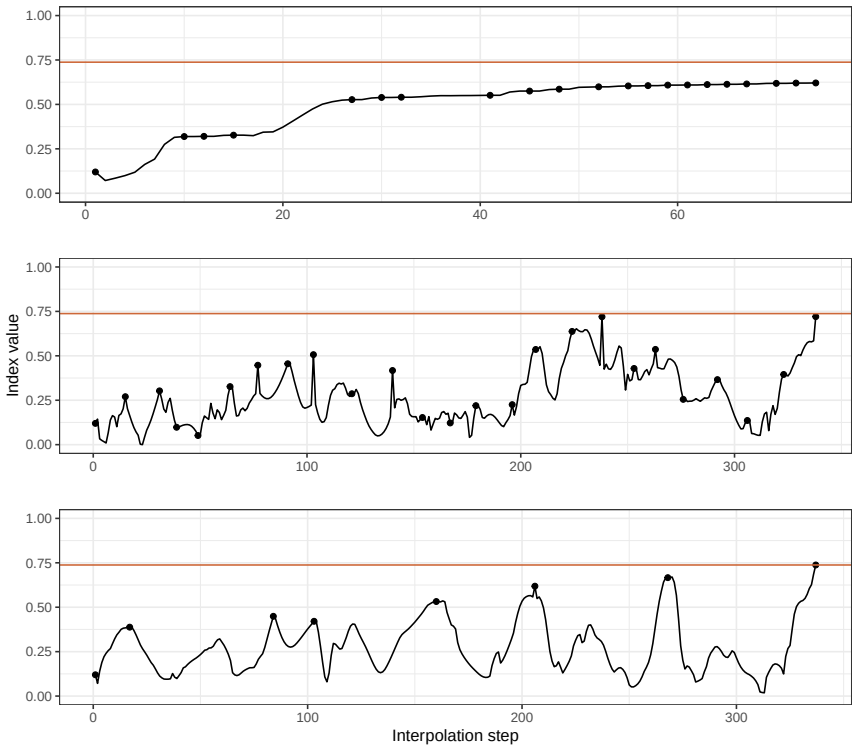

Figure 8: Splines index value along the interpolated optimization path in the guided tour using geodesic optimization (GEOO) (top), simulated annealing with geodesic interpolation (SAGEO) (middle) and with Givens interpolation (SAGIV) (bottom). Points indicate index values at target planes selected during the optimization, and the horizontal line shows the maximum across the three searches. With GEOO no within-plane rotation is possible and although the optimization always heads towards higher index values it finishes at a different frame in the same plane, with a lower than possible index value. With SAGEO, a different frame in the same plane interferes with the optimization. This does not happen with SAGIV, allowing the search to find the optimal view.

paths to a target *plane* and does not reach a specific target *frame*. By construction, this gives a smooth path. However, we can see that as a consequence, the optimization converges to a plane that does not identify the pattern of interest.

In the middle, we are showing the path found when using SAGEO. In this case, the search can suggest any target frame, also allowing for any within-plane rotation. The algorithm will compute the index value for the suggested target frame and accept it if the index value is larger than that for the current frame. However, once the target is accepted, the interpolation step will reset the orientation within the plane, potentially resulting in a frame that shows the same plane but with a lower index value. This is why we see drops in the index value even from one target plane to the next, and the optimization also does not identify the pattern of interest, even though it comes close.

Finally, in the bottom row, we see the result of the random search with Givens interpolation. While the index value can drop along an interpolation path, the value is strictly increasing for the target planes (up to small differences from numeric precision in the interpolation), as is required for the optimization. Note also that the search converged much faster (fewer target planes were selected). The length of the interpolated paths is similar because, for a fixed angular step size, the Givens interpolation will take more steps to interpolate between frames.

6 Conclusion

This paper describes the R package **woylier** which provides an implementation of the Givens interpolation path algorithm that can be used as an alternative interpolation method for a tour. The algorithm implemented in the **woylier** package is as described in Buja et al. (2005).

It is important to mention that **woylier** package should be used in conjunction with the **tourr** package. Currently, it is implemented for tours into 1D or 2D. A potential future development would be to generalize the interpolation for more than 2D projections of data, which could not be done here because it requires new mathematical derivations.

For most purposes, geodesic interpolation is desirable. The motivation for frame-to-frame interpolation arises from a rotational invariance problem of the current geodesic interpolation algorithm implemented in **tourr** package. Frame-to-frame interpolation provides users with the tools to move to a specified frame, not the plane it defines.

Our application shows the Givens interpolation used with a projection pursuit index, which is not rotationally invariant but ideal for detecting bivariate non-linear association in high-dimensional data. The potential applications where this might be useful are many and varied, possibly including facial recognition algorithms and morphing between images, or imputing missing time steps in high-dimensional time series.

Acknowledgements

A prior implementation using C was available in the XGobi software (Swayne et al., 1998). This new implementation was undertaken at the request of a **tourr** package user.

We thank Saurabh Jhanjee for his contributions at the start of the project. This article is created using **knitr** (Xie, 2022) and **rmarkdown** (Allaire et al., 2026; Xie et al., 2018, 2020) in R. For graphs we have used **ggplot2** (Wickham, 2016), **cowplot** (Wilke, 2020) and **GGally** (Schloerke et al., 2021), for tables we have used **kableExtra** (Zhu, 2021). The source code for reproducing this paper can be found at: https://github.com/uschiLaa/woylier-article.

Bibliography

J. Allaire, Y. Xie, C. Dervieux, J. McPherson, J. Luraschi, K. Ushey, A. Atkins, H. Wickham, J. Cheng, W. Chang, and R. Iannone. *rmarkdown: Dynamic Documents for R*, 2026. URL https://github.com/rstudio/rmarkdown. R package version 2.31. [p12]

D. Asimov. The grand tour: a tool for viewing multidimensional data. *SIAM J Sci Stat Comput 6(1)*, page 128–143, 1985. doi: 10.1137/0906011. [p1]

A. Buja, D. Cook, D. Asimov, and C. B. Hurley. Theory of dynamic projections in high-dimensional data visualization. 2004. URL http://stat.wharton.upenn.edu/~buja/PAPERS/paper-dyn-proj-math.pdf. [p1]

A. Buja, D. Cook, D. Asimov, and C. Hurley. Computational methods for high-dimensional rotations in data visualization. *Handbook of Statistics*, page 391–413, 2005. doi: 10.1016/s0169-7161(04)24014-7. [p1, 2, 11]

D. Cook and A. Buja. Manual controls for high-dimensional data projections. *Journal of Computational and Graphical Statistics*, 6(4):464–480, 1997. ISSN 1061-8600. doi: 10.2307/1390747. URL http://www.jstor.org/stable/1390747. [p1]

D. Cook, A. Buja, and J. Cabrera. Projection pursuit indexes based on orthonormal function expansions. *Journal of Computational and Graphical Statistics*, 2:225–250, 1993. doi: 10.1080/10618600.1993.10474610. URL https://doi.org/10.1080/10618600.1993.10474610. [p1]

D. Cook, A. Buja, J. Cabrera, and C. Hurley. Grand tour and projection pursuit. *Journal of Computational and Graphical Statistics*, 4(3):155–172, 1995. doi: 10.1080/10618600.1995.10474674. URL https://www.tandfonline.com/doi/abs/10.1080/10618600.1995.10474674. [p1]

Y. Duan and J. Cabrera. *PPbigdata: Projection Pursuit for Big Data Based on Data Nuggets*, 2024. URL https://CRAN.R-project.org/package=PPbigdata. R package version 1.0.0. [p1]

D. Fischer, A. Berro, K. Nordhausen, and A. Ruiz-Gazen. *REPPlab: An R package for detecting clusters and outliers using exploratory projection pursuit*, 2021. URL https://doi.org/10.1080/03610918.2019.1626880. [p1]

G. H. Golub and C. F. V. Loan. *Matrix computations (4th ed)*. Johns Hopkins University Press, 2013. ISBN 0801837723. doi: 10.1353/book.23351. [p4]

K. Grimm. *Kennzahlenbasierte Grafikauswahl*. doctoral thesis, Universität Augsburg, 2016. [p1]

D. P. Hofmeyr and N. G. Pavlidis. PPCI: an R package for cluster identification using projection pursuit. *The R Journal*, 2019. doi: 10.32614/RJ-2019-046. URL https://journal.r-project.org/archive/2019/RJ-2019-046/index.html. [p1]

J. Kruskal. Toward a practical method which helps uncover the structure of a set of observations by finding the line transformation which optimizes a new index of condensation. *Statistical computation; New York, Academic Press*, pages 427–440, 1969. doi: 10.1016/B978-0-12-498150-8.50024-0. [p1]

U. Laa and D. Cook. Using tours to visually investigate properties of new projection pursuit indexes with application to problems in physics. *Computational Statistics*, 35:1171–1205, 2020. doi: 10.1007/s00180-020-00954-8. [p1]

U. Laa, A. Aumann, D. Cook, and G. Valencia. New and simplified manual controls for projection and slice tours, with application to exploring classification boundaries in high dimensions. *Journal of Computational and Graphical Statistics*, 0(0):1–8, 2023. doi: 10.1080/10618600.2023.2206459. [p1]

S. Lee, D. Cook, N. da Silva, U. Laa, N. Spyrison, E. Wang, and H. S. Zhang. The state-of-the-art on tours for dynamic visualization of high-dimensional data. *WIREs Computational Statistics*, 14(4): e1573, 2022. doi: https://doi.org/10.1002/wics.1573. [p1]

P. C. Ossani and M. A. Cirillo. *Pursuit: Projection Pursuit*, 2026. URL https://CRAN.R-project.org/package=Pursuit. R package version 1.1.0. [p1]

O. E. Rates. Open exchange rates api, 2026. URL https://openexchangerates.org. Accessed: 2022-10-12. [p8]

B. Schloerke. *geozoo: Zoo of Geometric Objects*, 2016. URL https://CRAN.R-project.org/package=geozoo. R package version 0.5.1. [p8]

B. Schloerke, D. Cook, J. Larmarange, F. Briatte, M. Marbach, E. Thoen, A. Elberg, and J. Crowley. *GGally: Extension to 'ggplot2'*, 2021. URL https://CRAN.R-project.org/package=GGally. R package version 2.1.2. [p12]

L. Scrucca and A. Serafini. Projection pursuit based on gaussian mixtures and evolutionary algorithms. *Journal of Computational and Graphical Statistics*, 28(4):847—-860, 2019. doi: 10.1080/10618600.2019.1598871. [p1]

D. F. Swayne, D. Cook, and A. Buja. XGobi: Interactive dynamic data visualization in the x window system. *Journal of Computational and Graphical Statistics*, 7(1):113–130, 1998. doi: 10.1080/10618600.1998.10474764. URL https://github.com/ggobi/xgobi. [p12]

H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. ISBN 978-3-319-24277-4. URL https://ggplot2.tidyverse.org. [p12]

H. Wickham, D. Cook, H. Hofmann, and A. Buja. tourr: An R package for exploring multivariate data with projections. *Journal of Statistical Software*, 40(2):1–18, 2011. doi: 10.18637/jss.v040.i02. [p1, 5]

C. O. Wilke. *cowplot: Streamlined Plot Theme and Plot Annotations for 'ggplot2'*, 2020. URL https://CRAN.R-project.org/package=cowplot. R package version 1.1.1. [p12]

S. N. Wood. Fast stable restricted maximum likelihood and marginal likelihood estimation of semi-parametric generalized linear models. *Journal of the Royal Statistical Society (B)*, 73(1):3–36, 2011. doi: 10.1111/j.1467-9868.2010.00749.x. [p1]

Y. Xie. *knitr: A General-Purpose Package for Dynamic Report Generation in R*, 2022. URL https://yihui.org/knitr/. R package version 1.38. [p12]

Y. Xie, J. Allaire, and G. Grolemund. *R Markdown: The Definitive Guide*. Chapman and Hall/CRC, Boca Raton, Florida, 2018. ISBN 9781138359338. doi: 10.1201/9781138359444. URL https://yihui.org/rmarkdown/. [p12]

Y. Xie, C. Dervieux, and E. Riederer. *R Markdown Cookbook*. Chapman and Hall/CRC, Boca Raton, Florida, 2020. ISBN 9780367563837. doi: 10.1201/9781003097471. URL https://yihui.org/rmarkdown-cookbook. [p12]

H. S. Zhang, D. Cook, U. Laa, N. Langrené, and P. Menéndez. Visual diagnostics for constrained optimisation with application to guided tours. *The R Journal*, 13(2):624–641, 2021. doi: 10.32614/RJ-2021-105. URL https://huizezhang-sherry.github.io/ferrn/. [p9]

H. Zhu. *kableExtra: Construct Complex Table with 'kable' and Pipe Syntax*, 2021. URL https://CRAN.R-project.org/package=kableExtra. R package version 1.3.4. [p12]

Zoljargal Batsaikhan
Monash University
Department of Econometrics and Business Statistics
Clayton, VIC, Australia
https://github.com/zolabat
ORCiD: 0009-0005-0055-1448
zoljargal11@gmail.com

Dianne Cook
Monash University
Department of Econometrics and Business Statistics
Clayton, VIC, Australia
http://www.dicook.org
ORCiD: 0000-0002-3813-7155
dicook@monash.edu

Ursula Laa
University of Natural Resources and Life Sciences Vienna
Institute of Statistics
Vienna, Austria
https://uschilaa.github.io
ORCiD: 0000-0002-0249-6439
ursula.laa@boku.ac.at